DESARROLLO WEB EN ENTORNO SERVIDOR

PHP desde cero

De los fundamentos a PDO, sesiones y Docker

hola.php

```php
<?php
$nombre = 'mundo';
echo "Hola, $nombre";
// Hola, mundo
```

<?php

01

Qué es PHP

Definición y usos habituales



Definiciones: qué es PHP

Lenguaje de servidor

Se ejecuta en el servidor, no en el navegador. El usuario solo recibe el resultado: HTML, JSON o una imagen.

Lenguaje interpretado

El código se ejecuta directamente, línea a línea, sin un paso previo de compilación manual.

Página dinámica

Página que se genera en cada petición y puede cambiar según el usuario, la hora o los datos de una base de datos.

Tipado dinámico

Las variables no declaran su tipo: PHP lo deduce del valor que guardan y puede cambiar.

Open source

Código abierto y gratuito: cualquiera puede usarlo, estudiarlo y modificarlo.

CMS y framework

Un CMS (WordPress) es una aplicación lista para gestionar contenido. Un framework (Laravel) es una base de código para construir la tuya.

¿Qué es PHP?

PHP: Hypertext Preprocessor

Un lenguaje de programación que se ejecuta en el servidor y genera páginas web dinámicas.

- **Lado servidor**
- **Interpretado**
- **Open source**
- **Tipado dinámico**

Navegador
pide /perfil.php

→

Apache + PHP
ejecuta el código

→

Base de datos
aporta los datos

→

Navegador
recibe solo HTML

El usuario nunca ve el código PHP: solo el resultado que genera

Usos habituales de PHP



CMS

WordPress, Drupal o Joomla. WordPress mueve más del 40 % de las webs.



Frameworks

Laravel y Symfony para aplicaciones web completas.



Tiendas online

WooCommerce, PrestaShop y Magento.



APIs REST

Backends que devuelven JSON a apps web y móviles.



Wikis y portales

Wikipedia funciona con MediaWiki, escrito en PHP.



Intranets y paneles

Herramientas internas, gestión y administración.

Puntos fuertes y críticas



Puntos fuertes

- Fácil de empezar: un archivo y ya funciona
- Hosting barato y disponible en todas partes
- Ecosistema maduro: Composer, Laravel, Symfony
- Muy rápido desde PHP 7 y 8



Críticas habituales

- Mala fama por código antiguo y desordenado
- Nombres de funciones poco consistentes
- Permite malas prácticas si no se tiene cuidado
- La seguridad depende de quien programa

02

Entorno de trabajo

Cómo ejecutar PHP y cómo preparar VS Code



Definiciones: entorno de trabajo

Servidor web

Programa que recibe peticiones HTTP y devuelve respuestas. Apache es el más habitual con PHP.

Intérprete de PHP

Programa que lee y ejecuta los archivos .php. Se comprueba la versión con php -v.

CLI

Command Line Interface: ejecutar PHP desde la terminal (php archivo.php), sin navegador.

localhost y puerto

localhost es tu propio ordenador. El puerto (8080) indica a qué programa va dirigida la petición.

IDE / editor

Programa para escribir código. VS Code es un editor que se amplía con extensiones.

Extensión

Complemento que añade funciones al editor, como autocompletado de PHP (Intelephense) o depuración.

Cuatro formas de ejecutar PHP



Recomendado

Docker

Misma versión para toda la clase. Web y base de datos en contenedores.

```
docker compose up -d
```



XAMPP / MAMP

Paquete con Apache, PHP y MySQL. Fácil, pero distinto en cada equipo.

```
Panel de control
```



PHP instalado

Con Homebrew en Mac o winget en Windows. Ideal para la terminal.

```
brew install php
```



Servidor integrado

PHP trae un servidor web de desarrollo. Nunca para producción.

```
php -S localhost:8000
```

VS Code: extensiones recomendadas



PHP Intelephense

Autocompletado, errores, navegación y formato.



PHP Debug

Depurar con Xdebug: puntos de ruptura y variables.



Container Tools

Ver y gestionar contenedores y compose.yaml.



Error Lens

Muestra los errores en la misma línea del código.



GitLens

Historial y autoría de cada línea con Git.



Tema e iconos

One Dark Pro o GitHub Theme y Material Icon Theme.

Tu primer archivo y cómo ejecutarlo

hola.php

```php
<?php
$curso = '2DAW';
echo "Hola, $curso";
```

Navegador

`localhost:8000/hola.php`

Hola, 2DAW

Terminal (CLI) `php hola.php`

Muestra «Hola, 2DAW» en la terminal

Servidor integrado `php -S localhost:8000`

Abre localhost:8000/hola.php

Docker `docker compose up -d`

Abre localhost:8080/hola.php

Comprueba la versión con php -v

03

Fundamentos

Sintaxis, variables, tipos, cadenas y operadores



Definiciones: fundamentos

Variable

Espacio con nombre que guarda un valor que puede cambiar. En PHP empieza por $: $edad.

Constante

Valor con nombre que no cambia durante la ejecución. Se define con const y sin $.

Tipo de dato

Clase de valor que guarda una variable: string, int, float, bool, array o null.

Cadena (string)

Secuencia de caracteres entre comillas: 'Hola' o "Hola, $nombre".

Concatenar e interpolar

Concatenar es unir textos con el punto (.). Interpolar es meter una variable dentro de comillas dobles.

Operador

Símbolo que realiza una operación: aritmético (+), de comparación (===) o lógico (&&).

Sintaxis básica

sintaxis.php

```php
<?php
// Comentario de una línea
# También vale con almohadilla
/* Comentario
   de varias líneas */

echo 'Hola';     // imprime texto
print 'Adiós';   // casi igual que echo
?>
<h1><?= 'Título desde PHP' ?></h1>
<p>Esto es HTML normal</p>
```

`<?php … ?>`	Delimita el código PHP dentro del archivo
`;`	Cada instrucción termina en punto y coma
`echo`	Envía texto a la respuesta
`<?= … ?>`	Forma corta de echo dentro del HTML
`// # /* */`	Comentarios: no se ejecutan

Paso a paso: cómo llega el texto al navegador

QUÉ ES

echo envía texto a la respuesta. Lo que imprime echo, junto con el HTML que hay fuera de <?php ?>, forma la página que recibe el navegador.

CÓMO FUNCIONA

1. El navegador pide saludo.php al servidor
2. PHP ejecuta el archivo de arriba abajo
3. Cada echo y cada trozo de HTML se van añadiendo a la respuesta
4. El navegador recibe solo el resultado y lo pinta

ERROR TÍPICO

Olvidar el punto y coma al final de una instrucción. PHP da «syntax error» y suele señalar la línea siguiente, no la que tiene el fallo.

saludo.php

```
<h1>Bienvenida</h1>
<?php
$hora = 9;
echo '<p>Son las ' . $hora . '</p>';
?>
<p>Fin</p>
```

SALIDA REAL

```
<h1>Bienvenida</h1>
<p>Son las 9</p><p>Fin</p>
```

Variables y tipos de datos

variables.php

```php
<?php
$nombre  = 'Ana';          // string
$edad    = 19;             // int
$nota    = 8.5;            // float
$aprobado = true;          // bool
$movil   = null;           // null
$modulos = ['DWES', 'DWEC']; // array

var_dump($nota);
// float(8.5)
echo gettype($edad);
// integer
```

string

'Ana'

Texto

int

19

Entero

float

8.5

Decimal

bool

true

Verdadero / falso

null

null

Sin valor

array

['DWES']

Colección

Paso a paso: tipado dinámico y var_dump

QUÉ ES

En PHP una variable no tiene un tipo fijo: el tipo lo marca el valor que guarda en cada momento. var_dump muestra el tipo y el valor.

CÓMO FUNCIONA

1. $x = 5 guarda un int
2. $x = 'cinco' cambia la misma variable a string
3. Al operar, PHP convierte si puede: '5' + 3 da 8
4. Con el punto, PHP une textos: '5' . 3 da "53"

ERROR TÍPICO

Sumar un texto que no es número: 'abc' + 1 no da 'abc1'. En PHP 8 detiene el programa con «TypeError: Unsupported operand types».

ejemplo

```php
$x = 5;
var_dump($x);
$x = 'cinco';
var_dump($x);
var_dump('5' + 3);
var_dump('5' . 3);
```

SALIDA REAL

```
int(5)
string(5) "cinco"
int(8)
string(2) "53"
```

Cadenas de texto

Con `$nombre = 'Ana'`

```
'Hola, $nombre'
```
→ **Hola, $nombre**

Comillas simples: texto literal

```
"Hola, $nombre"
```
→ **Hola, Ana**

Comillas dobles: interpolan la variable

```
'Hola, ' . $nombre
```
→ **Hola, Ana**

El punto concatena

Función	Qué hace
`strlen($s)`	Longitud en bytes
`strtoupper($s)`	Pasa a mayúsculas
`trim($s)`	Quita espacios al inicio y al final
`str_contains($s, $b)`	¿Contiene el texto?
`str_replace($a, $b, $s)`	Sustituye texto
`number_format($n, 2)`	Formatea un número

Paso a paso: concatenar e interpolar

QUÉ ES

Concatenar es unir trozos de texto con el punto. Interpolar es escribir la variable dentro de comillas dobles para que PHP ponga su valor.

CÓMO FUNCIONA

1. PHP lee la cadena de izquierda a derecha
2. Con comillas dobles, sustituye cada $variable por su valor
3. Con comillas simples, deja el texto tal cual, sin sustituir
4. Para un array dentro de comillas, usa llaves: {$a['nombre']}

ERROR TÍPICO

Olvidar los espacios: 'Hola' . $nombre da «HolaAna». El espacio va dentro de las comillas: 'Hola ' . $nombre.

ejemplo

```php
$a = ['nombre' => 'Ana', 'nota' => 8.5];

echo 'Alumna: ' . $a['nombre'] . "\n";
echo "Alumna: {$a['nombre']}\n";
echo "Nota: {$a['nota']}";
```

SALIDA REAL

```
Alumna: Ana
Alumna: Ana
Nota: 8.5
```

Operadores

Aritméticos

`+` `-` `*` `/` `%` `**`

Suma, resta, producto, división, resto y potencia

Comparación

`==` `===` `!==` `<` `>` `<=>`

Comparan valores; === también el tipo

Lógicos

`&&` `||` `!`

Y, o, negación

Null

`??` `??=`

Valor por defecto si no existe o es null

`'1' == 1` → true `'1' === 1` → false (distinto tipo) `$_GET['p'] ?? 1` → 1 si no llega

Paso a paso: == frente a ===

QUÉ ES

== compara el valor después de convertir los tipos. === compara valor y tipo, sin convertir nada. Por eso === es más seguro y predecible.

CÓMO FUNCIONA

1. '1' == 1: PHP convierte '1' en número y compara: true
2. '1' === 1: string frente a int: false, sin mirar el valor
3. null == false es true, pero null === false es false
4. Regla: usa === y !== salvo que quieras convertir a propósito

ERROR TÍPICO

Comprobar strpos con ==: strpos('Hola', 'H') devuelve 0 (está en la posición 0) y 0 == false es true. Hay que usar !== false.

ejemplo

```php
var_dump('1' == 1);
var_dump('1' === 1);
var_dump(0 == 'a');
var_dump(null == false);
var_dump(null === false);
var_dump(strpos('Hola', 'H') == false);
```

SALIDA REAL

```
bool(true)
bool(false)
bool(false)
bool(true)
bool(false)
bool(true)
```

04

Control de flujo

Condicionales, bucles, arrays, foreach y funciones



Definiciones: control de flujo

Condicional

Estructura que ejecuta un bloque solo si se cumple una condición: if, else if, switch, match.

Bucle

Estructura que repite un bloque mientras se cumple una condición o para cada elemento: for, while, foreach.

Array indexado

Lista de valores ordenados por posición, empezando en 0: $modulos[0].

Array asociativo

Colección de pares clave → valor: $alumno['nombre'].

Función

Bloque de código con nombre que se puede reutilizar. Recibe parámetros y puede devolver un valor.

Parámetro y retorno

El parámetro es el dato que recibe la función. El retorno (return) es el resultado que devuelve.

Clase

Plantilla que define datos (propiedades) y acciones (métodos).

Objeto

Ejemplar creado a partir de una clase con new. Se usa con ->: $pdo->prepare().

Condicionales: if, elseif y else

notas.php

```php
$nota = 7.5;

if ($nota >= 9) {
    $txt = 'Sobresaliente';
} elseif ($nota >= 7) {
    $txt = 'Notable';
} elseif ($nota >= 5) {
    $txt = 'Aprobado';
} else {
    $txt = 'Suspenso';
}
```

ternario

```php
$estado = $nota >= 5 ? 'Apto' : 'No apto';
```

Qué bloque se ejecuta según la nota



Claves

- Condición entre paréntesis, bloque entre llaves
- Solo se ejecuta un bloque
- Ordena de lo más restrictivo a lo menos
- Prueba los valores frontera: 5, 7 y 9

Paso a paso: condiciones compuestas

QUÉ ES

&& (y) exige que se cumplan todas las condiciones; || (o) basta con que se cumpla una; ! (no) invierte el resultado.

CÓMO FUNCIONA

1. true && false da false: con && basta un false
2. true || false da true: con || basta un true
3. PHP deja de evaluar en cuanto sabe el resultado
4. Los paréntesis marcan el orden: a && (b || c)

ERROR TÍPICO

Escribir un rango imposible: $nota >= 5 && $nota < 5 nunca es true. Para «fuera de rango» se usa ||: $nota < 0 || $nota > 10.

ejemplo

```php
$edad = 17;
$entrada = false;
$invitado = true;

if ($edad >= 16 && ($entrada || $invitado)) {
    echo 'Puede pasar';
} else {
    echo 'No puede pasar';
}
if (!$entrada) {
    echo "\nSin entrada comprada";
}
```

SALIDA REAL

```
Puede pasar
Sin entrada comprada
```

switch y match

switch · clásico

```php
switch ($dia) {
    case 'sab':
    case 'dom':
        $tipo = 'Fin de semana';
        break;
    default:
        $tipo = 'Laborable';
}
```

match · PHP 8

```php
$tipo = match ($dia) {
    'sab', 'dom' => 'Fin de semana',
    default      => 'Laborable',
};
```

Compara con == · necesita break · es una sentencia

Compara con === · sin break · devuelve un valor

Bucles: for, while y do-while

for Sabes cuántas vueltas

for.php

```php
for ($i = 1; $i <= 3; $i++) {
    echo $i;
}
// 123
```

while Mientras se cumpla

while.php

```php
$n = 3;
while ($n > 0) {
    echo $n--;
}
// 321
```

do-while Al menos una vez

do-while.php

```php
$n = 0;
do {
    echo $n;
} while ($n > 0);
// 0
```

break Sale del bucle por completo

continue Salta a la siguiente vuelta

Paso a paso: el bucle for

QUÉ ES

for repite un bloque un número conocido de veces. Tiene tres partes separadas por punto y coma: inicio, condición y actualización.

CÓMO FUNCIONA

1. Inicio: $i = 1 se ejecuta una sola vez
2. Condición: si $i <= 3 es true, entra en el bloque
3. Al terminar el bloque se ejecuta $i++
4. Vuelve a la condición; cuando es false, sale del bucle

ERROR TÍPICO

Una actualización que no acerca al final: con $i-- en lugar de $i++, la condición $i <= 3 siempre es true y el bucle no termina.

ejemplo

```php
for ($i = 1; $i <= 3; $i++) {
    echo "Vuelta $i\n";
}
echo "Al salir, \$i vale $i";
```

SALIDA REAL

```
Vuelta 1
Vuelta 2
Vuelta 3
Al salir, $i vale 4
```

Arrays: indexados, asociativos y anidados

Indexado

indexado.php

```php
$modulos = ['DWES', 'DWEC', 'DAW'];
echo $modulos[0]; // DWES
$modulos[] = 'EIE'; // añade
```

Asociativo

asociativo.php

```php
$alumno = [
    'nombre' => 'Ana',
    'nota'   => 8.5,
];
echo $alumno['nombre'];
```

Anidado

anidado.php

```php
$clase = [
    ['nombre' => 'Ana',
     'nota'   => 8.5],
    ['nombre' => 'Leo',
     'nota'   => 6],
];
echo $clase[1]['nombre']; // Leo
```

Funciones que usarás mucho

`count()` `in_array()` `array_keys()` `sort()` `array_map()` `array_filter()`

Paso a paso: claves e índices

QUÉ ES

Un array de PHP es siempre una lista de pares clave → valor. En un array indexado, las claves son números que pone PHP empezando en 0.

CÓMO FUNCIONA

1. ['DWES', 'DWEC'] guarda 0 => 'DWES' y 1 => 'DWEC'
2. $m[] = 'DAW' añade con la siguiente clave libre: 2
3. Si fuerzas $m[10], el siguiente [] usará la clave 11
4. En un asociativo eres tú quien elige las claves

ERROR TÍPICO

Contar desde 1: en ['DWES', 'DWEC'], $m[2] no existe y PHP avisa con «Undefined array key 2». El último índice es count($m) - 1.

ejemplo

```php
$m = ['DWES', 'DWEC'];
$m[] = 'DAW';
echo implode(', ', array_keys($m)), "\n";

$m[10] = 'EIE';
$m[] = 'IPE';
echo implode(', ', array_keys($m)), "\n";
echo $m[11];
```

SALIDA REAL

```
0, 1, 2
0, 1, 2, 10, 11
IPE
```

Funciones de arrays (1/2): consultar

`count()`

Devuelve cuántos elementos tiene un array.

ejemplo

```php
$modulos = ['DWES', 'DWEC',
            'DAW'];
echo count($modulos);
// 3
```

Ojo: un array vacío da 0. Sirve para saber si hay resultados: if (count($lista) > 0).

`in_array()`

Comprueba si un valor está dentro del array. Devuelve true o false.

ejemplo

```php
$modulos = ['DWES', 'DWEC'];
in_array('DWEC', $modulos);
// true
in_array('dwec', $modulos);
// false
```

Ojo: distingue mayúsculas. Sin el tercer parámetro true, '5' y 5 cuentan como iguales.

`array_keys()`

Devuelve una lista con las claves del array.

ejemplo

```php
$alumno = ['nombre' => 'Ana',
           'nota'   => 8.5];
array_keys($alumno);
// ['nombre', 'nota']
```

Ojo: devuelve las claves, no los valores. Para los valores existe array_values().

Funciones de arrays (2/2): ordenar y transformar

`sort()`

Ordena los valores de menor a mayor. Cambia el propio array.

ejemplo

```php
$notas = [7, 3, 9];
sort($notas);
// $notas = [3, 7, 9]
```

Ojo: devuelve true, no el array: `$x = sort($notas)` no sirve. `rsort()` ordena al revés.

`array_map()`

Aplica una función a cada elemento y devuelve un array nuevo con los resultados.

ejemplo

```php
$nums = [1, 2, 3];
$dobles = array_map(
    fn($n) => $n * 2, $nums);
// [2, 4, 6]
```

Ojo: primero va la función y después el array. El original no cambia.

`array_filter()`

Devuelve solo los elementos que cumplen una condición.

ejemplo

```php
$notas = [4, 7, 9, 3];
$aprobadas = array_filter(
    $notas,
    fn($n) => $n >= 5);
// [1 => 7, 2 => 9]
```

Ojo: aquí el array va primero. Conserva las claves (1 y 2); `array_values()` las renumera desde 0.

foreach: recorrer colecciones

solo valores

```php
foreach ($modulos as $modulo) {
    echo $modulo;
}
```

clave => valor

```php
foreach ($alumno as $campo => $valor) {
    echo "$campo: $valor";
}
```

generar una tabla

```php
<?php foreach ($clase as $a): ?>
  <tr><td><?= htmlspecialchars($a['nombre']) ?></td>
      <td><?= $a['nota'] ?></td></tr>
<?php endforeach; ?>
```

Resultado en el navegador

Nombre	Nota
Ana	8.5
Leo	6
Marta	9

La sintaxis alternativa foreach (…): … endforeach; es más legible cuando el bucle está mezclado con HTML.

Paso a paso: foreach

QUÉ ES

foreach recorre un array elemento a elemento, sin contador. En cada vuelta copia la clave y el valor actuales en las variables que indicas.

CÓMO FUNCIONA

1. Vuelta 1: $nombre = 'Ana', $nota = 8.5
2. Vuelta 2: $nombre = 'Leo', $nota = 4
3. Vuelta 3: $nombre = 'Marta', $nota = 9
4. No quedan elementos: el bucle termina solo

ERROR TÍPICO

Cambiar la variable del bucle esperando cambiar el array: $nota = 10 no toca $notas. Para eso hay que usar la clave: $notas[$nombre] = 10.

ejemplo

```php
$notas = ['Ana' => 8.5, 'Leo' => 4, 'Marta' => 9];

foreach ($notas as $nombre => $nota) {
    $estado = $nota >= 5 ? 'aprobado' : 'suspenso';
    echo "$nombre: $estado\n";
}
```

SALIDA REAL

```
Ana: aprobado
Leo: suspenso
Marta: aprobado
```

Funciones

funciones.php

```php
<?php
declare(strict_types=1);

function media(array $notas, int $decimales = 2): float
{
    $suma = array_sum($notas);
    return round($suma / count($notas), $decimales);
}

echo media([7, 8.5, 9]);              // 8.17
echo media([7, 8.5, 9], decimales: 0); // 8

// Función flecha (arrow function)
$doble = fn(int $n): int => $n * 2;
```

`array $notas`

Parámetro con tipo

`int $decimales = 2`

Valor por defecto

`: float`

Tipo que devuelve

`decimales: 0`

Argumento con nombre

`fn() =>`

Función flecha de una línea

Paso a paso: ámbito de las variables

QUÉ ES

Las variables de dentro de una función son locales: no ven las de fuera y desaparecen al salir. Los datos entran por parámetros y salen con return.

CÓMO FUNCIONA

1. Al llamar, el parámetro recibe una copia del valor
2. Dentro solo existen los parámetros y sus variables propias
3. return entrega el resultado y la función termina
4. Fuera, $precio sigue valiendo lo mismo que antes

ERROR TÍPICO

Usar dentro una variable de fuera: return $precio * (1 + $iva); avisa «Undefined variable $iva» y calcula 10. La solución es pasarla como parámetro.

ejemplo

```php
$iva = 0.21;

function conIva(float $precio): float
{
    // Aquí dentro $iva no existe
    return $precio * 1.21;
}

$precio = 10;
echo conIva($precio), "\n";
echo $precio;
```

SALIDA REAL

```
12.1
10
```

Organizar el código en archivos

proyecto

```
src/
├── index.php
├── productos.php
└── inc/
    ├── db.php
    ├── funciones.php
    └── cabecera.php
```

productos.php

```php
require_once __DIR__ . '/inc/db.php';
require_once __DIR__ . '/inc/funciones.php';
include __DIR__ . '/inc/cabecera.php';
```

`require`

Si falta el archivo: error fatal y se detiene

`include`

Si falta: aviso y el script continúa

`require_once`

Como require, pero solo carga una vez

`include_once`

Como include, pero solo carga una vez

Lo mínimo de objetos para entender PDO

Alumno.php

```php
class Alumno
{
    public function __construct(
        public string $nombre,
        private float $nota,
    ) {}

    public function aprobado(): bool
    {
        return $this->nota >= 5;
    }
}

$ana = new Alumno('Ana', 8.5);
echo $ana->nombre;      // Ana
var_dump($ana->aprobado()); // true
```

`class`	El molde: define datos y comportamiento
`new`	Crea un objeto a partir de la clase
`->`	Usa un método o propiedad del objeto
`::`	Accede a constantes: PDO::FETCH_ASSOC

Con PDO:

```php
$pdo = new PDO(…);
$pdo->prepare(…);
```

05

Formularios y sesiones

Recibir datos del usuario y recordarlo entre páginas



Definiciones: formularios y sesiones

Formulario

Elemento HTML <form> que permite al usuario enviar datos al servidor.

GET

Método HTTP que envía los datos en la URL (?id=5). Para consultar, nunca para datos privados.

POST

Método HTTP que envía los datos en el cuerpo de la petición. Para enviar o modificar información.

Superglobal

Array que PHP rellena automáticamente en cada petición: $_GET, $_POST, $_SESSION...

Validar

Comprobar que un dato cumple las reglas esperadas antes de usarlo: no vacío, email correcto, número en rango.

Escapar (XSS)

Convertir caracteres especiales para que se muestren como texto y no se ejecuten como HTML o JavaScript.

Cookie

Pequeño dato que el servidor guarda en el navegador y que este reenvía en cada petición.

Sesión

Datos de un usuario guardados en el servidor y asociados a un identificador que viaja en una cookie.

Las superglobales



$_GET

Datos de la URL: ?categoria=libros



$_POST

Datos de un formulario con method="post"



$_SESSION

Datos del usuario que se mantienen entre páginas



$_COOKIE

Cookies que envía el navegador



$_SERVER

Información de la petición: método, ruta, IP



$_FILES

Archivos subidos desde un formulario

Paso a paso: GET frente a POST

QUÉ ES

Son los dos métodos con los que un formulario envía datos. GET los pone en la URL; POST los envía en el cuerpo de la petición, sin que se vean en la barra.

CÓMO FUNCIONA

1. method="get": la URL queda buscar.php?q=php
2. PHP recoge el dato en $_GET['q']
3. method="post": la URL no cambia
4. PHP recoge el dato en $_POST['q']

ERROR TÍPICO

Enviar contraseñas o datos privados con GET: quedan en la URL, en el historial y en los registros del servidor. Para enviar o modificar datos, POST.

ejemplo

```
<form method="get" action="buscar.php">
  <input name="q">
  <button>Buscar</button>
</form>

<?php
// buscar.php?q=php
$q = $_GET['q'] ?? '';
echo 'Has buscado: ' . htmlspecialchars($q);
```

SALIDA REAL

```
Has buscado: php
```

Procesar un formulario con seguridad

contacto.php

```php
<?php
$errores = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nombre = trim($_POST['nombre'] ?? '');
    $email  = trim($_POST['email'] ?? '');

    if ($nombre === '') {
        $errores[] = 'El nombre es obligatorio';
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errores[] = 'El email no es válido';
    }
}
?>
<form method="post">
  <input name="nombre"> <input name="email">
  <button>Enviar</button>
</form>
```

1

Comprueba el método

`$_SERVER['REQUEST_METHOD']`

2

Limpia y valida

`trim()` · `filter_var()`

3

Escapa al mostrar

`htmlspecialchars()`

Paso a paso: validar y escapar

QUÉ ES

Validar es decidir si un dato es aceptable antes de usarlo. Escapar es convertirlo al mostrarlo, para que el navegador lo trate como texto y no como código.

CÓMO FUNCIONA

1. Llega $_POST['nombre'] con el valor Ana
2. Validar: no está vacío y no supera 50 caracteres
3. Escapar al mostrar: < pasa a < y > pasa a >
4. El navegador enseña Ana como texto, sin negrita

ERROR TÍPICO

Escapar al guardar en lugar de al mostrar. En la base de datos va el dato original; htmlspecialchars se aplica justo en el echo.

ejemplo

```php
$nombre = trim('<b>Ana</b>');

if ($nombre === '' || mb_strlen($nombre) > 50) {
    exit('Nombre no válido');
}
echo 'Sin escapar: ' . $nombre . "\n";
echo 'Escapado: ' . htmlspecialchars($nombre);
```

SALIDA REAL

```
Sin escapar: <b>Ana</b>
Escapado: &lt;b&gt;Ana&lt;/b&gt;
```

Sesiones: dar memoria a HTTP



Navegador

`PHPSESSID = a1b2c3…`

Solo guarda el identificador

envía la cookie en cada petición

responde con la página



Servidor PHP

```
sesión a1b2c3…

usuario => 'ana'
rol     => 'profesora'
```

1. session_start() crea un id y envía la cookie
2. El navegador devuelve la cookie en cada petición
3. PHP recupera los datos en $_SESSION

Sesiones en código

Guardar

login.php

```php
session_start();

$_SESSION['usuario'] = 'ana';
$_SESSION['rol'] = 'profe';
```

Leer

panel.php

```php
session_start();

$u = $_SESSION['usuario']
     ?? 'invitado';
echo "Hola, $u";
```

Cerrar

logout.php

```php
session_start();
$_SESSION = [];
session_destroy();
header('Location: login.php');
exit;
```



session_start() siempre al principio, antes de cualquier echo o HTML. Si no: «headers already sent».

Paso a paso: por qué session_start va lo primero

QUÉ ES

session_start() envía una cabecera HTTP con la cookie de sesión. Las cabeceras viajan antes que el contenido, así que no puede haberse impreso nada antes.

CÓMO FUNCIONA

1. Una respuesta HTTP lleva primero cabeceras y luego cuerpo
2. El primer echo, espacio o HTML empieza el cuerpo
3. A partir de ahí ya no se pueden enviar cabeceras
4. session_start(), header() y setcookie() dejan de funcionar

ERROR TÍPICO

Un espacio antes de <?php ya es salida. PHP avisa: «Session cannot be started after headers have already been sent».

visitas.php

```php
<?php
// Correcto: antes de cualquier salida
session_start();
$_SESSION['visitas'] =
    ($_SESSION['visitas'] ?? 0) + 1;
?>
<!doctype html>
<p>Visitas: <?= $_SESSION['visitas'] ?></p>
```

SALIDA REAL · RECARGANDO 3 VECES

```
Visitas: 1
Visitas: 2
Visitas: 3
```

Un login seguro con sesiones

login.php

```php
session_start();
// $usuario viene de la base de datos (PDO)
if ($usuario && password_verify($pass, $usuario['hash'])) {
    session_regenerate_id(true);
    $_SESSION['usuario_id'] = $usuario['id'];
    header('Location: panel.php');
    exit;
}
$error = 'Usuario o contraseña incorrectos';
```

proteger una página

```php
session_start();
if (!isset($_SESSION['usuario_id'])) {
    header('Location: login.php'); exit;
}
```



`password_hash()`

Al registrar: guarda el hash, nunca la contraseña



`password_verify()`

Al entrar: compara con el hash guardado



`session_regenerate_id`

Nuevo identificador tras el login



`Mensaje genérico`

No digas si falló el usuario o la contraseña

Paso a paso: qué es un hash

QUÉ ES

Un hash es una huella de la contraseña: se calcula en un solo sentido y no se puede deshacer. Se guarda la huella, nunca la contraseña.

CÓMO FUNCIONA

1. Registro: password_hash() calcula la huella
2. Se guarda solo la huella en la base de datos
3. Login: password_verify() compara lo escrito con la huella
4. Si roban la base de datos, no ven las contraseñas

ERROR TÍPICO

Comparar hashes con ===: password_hash da un resultado distinto cada vez, porque añade una «sal» aleatoria. Hay que usar password_verify.

ejemplo

```php
$hash = password_hash('secreto123',
                      PASSWORD_DEFAULT);
echo $hash, "\n";

var_dump(password_verify('secreto123', $hash));
var_dump(password_verify('Secreto123', $hash));
```

SALIDA REAL

```
$2y$10$FXYBq.tzMQa8oSQsO3I5YOaZONPr…
bool(true)
bool(false)
```

Cookies frente a sesiones

	Cookie	Sesión
Dónde se guarda	En el navegador	En el servidor
¿La ve el usuario?	Sí, y puede modificarla	No, solo su identificador
Duración	La que fijes con setcookie()	Hasta cerrar el navegador o caducar
Tamaño	Unos 4 KB	Sin límite práctico
Uso típico	Idioma, tema, «recordarme»	Usuario identificado, carrito
En PHP	setcookie() · $_COOKIE	session_start() · $_SESSION

06

Bases de datos con PDO

Conectar, consultar y escribir de forma segura



Definiciones: bases de datos con PDO

Base de datos

Sistema que almacena información organizada en tablas con filas y columnas: MariaDB, MySQL, PostgreSQL.

SQL

Lenguaje para consultar y modificar bases de datos: SELECT, INSERT, UPDATE, DELETE.

PDO

PHP Data Objects: la forma estándar de PHP para conectarse a cualquier base de datos con el mismo código.

DSN

Cadena que indica a qué base conectarse: driver, servidor, nombre de la base y codificación.

Driver

Extensión que permite a PDO hablar con un motor concreto: pdo_mysql, pdo_pgsql, pdo_sqlite.

Sentencia preparada

Consulta con marcadores en lugar de los datos (?, :nombre). Los datos se envían aparte con execute.

Inyección SQL

Ataque que mete código SQL dentro de un dato para alterar la consulta. Se evita con sentencias preparadas.

CRUD

Las cuatro operaciones básicas sobre los datos: Create, Read, Update y Delete.

¿Qué es PDO?



El DSN indica a qué base conectarse

```
mysql:host=db;dbname=tienda;charset=utf8mb4
```

driver | servidor | base de datos | codificación

Conectar con PDO

db.php

```php
<?php
$dsn  = 'mysql:host=db;dbname=tienda;charset=utf8mb4';
$user = 'app';
$pass = 'secreto'; // mejor: getenv(), ver bloque Docker

try {
    $pdo = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
} catch (PDOException $e) {
    error_log($e->getMessage());
    http_response_code(500);
    exit('No se pudo conectar con la base de datos');
}
```

`new PDO(…)`

Crea la conexión

`ERRMODE_EXCEPTION`

Los errores lanzan excepciones

`FETCH_ASSOC`

Filas como arrays asociativos

`try / catch`

Captura el fallo sin mostrar detalles

Leer datos y mostrarlos

productos.php

```php
require_once __DIR__ . '/inc/db.php';

$productos = $pdo
    ->query('SELECT nombre, precio FROM productos')
    ->fetchAll();
?>
<ul>
<?php foreach ($productos as $p): ?>
  <li><?= htmlspecialchars($p['nombre']) ?>
      · <?= $p['precio'] ?> €</li>
<?php endforeach; ?>
</ul>
```

Resultado

- Teclado · 29.90 €
- Ratón · 14.50 €
- Monitor · 159.00 €

fetchAll() devolvió:

```
[ ['nombre' => 'Teclado',
   'precio' => '29.90'], … ]
```

Los DECIMAL llegan como string.

`fetchAll()`

Todas las filas

`fetch()`

Una fila o false

`fetchColumn()`

Un único valor

Sentencias preparadas contra la inyección SQL

✕ **Inseguro: el dato se pega dentro del SQL**

NUNCA

```php
$email = $_POST['email'];
$sql = "SELECT * FROM usuarios
        WHERE email = '$email'";
$pdo->query($sql);
```

✓ **Seguro: prepare + execute con parámetros**

SIEMPRE

```php
$stmt = $pdo->prepare(
    'SELECT * FROM usuarios WHERE email = :email'
);
$stmt->execute(['email' => $_POST['email']]);
$usuario = $stmt->fetch();
```

Si alguien escribe en el campo email:

```
' OR '1'='1
```

→ `WHERE email = '' OR '1'='1'` → devuelve todos los usuarios

Paso a paso: prepare y execute

QUÉ ES

Una sentencia preparada separa la consulta de los datos. La base de datos recibe primero la estructura y después los valores, que nunca se interpretan como SQL.

CÓMO FUNCIONA

1. prepare(): envía la consulta con marcadores (:email)
2. La base de datos la analiza y la deja preparada
3. execute(): envía el valor de cada marcador
4. fetch() o fetchAll(): recoges el resultado

ERROR TÍPICO

Poner comillas al marcador: WHERE email = ':email' busca el texto literal «:email». El marcador se escribe sin comillas.

ejemplo

```php
$stmt = $pdo->prepare(
    'SELECT id, email FROM usuarios
     WHERE email = :email'
);

// El ataque llega como un dato más
$stmt->execute(['email' => "' OR '1'='1"]);
var_dump($stmt->fetch());
```

SALIDA REAL

```
bool(false)
```

Insertar, actualizar y borrar

crear — INSERT

```php
$st = $pdo->prepare(
  'INSERT INTO productos (nombre, precio)
   VALUES (:n, :p)');
$st->execute(['n' => $n, 'p' => $p]);
$id = $pdo->lastInsertId();
```

leer — SELECT

```php
$st = $pdo->prepare(
  'SELECT * FROM productos WHERE id = ?');
$st->execute([$id]);
$producto = $st->fetch();
```

modificar — UPDATE

```php
$st = $pdo->prepare(
  'UPDATE productos SET precio = ?
   WHERE id = ?');
$st->execute([$precio, $id]);
echo $st->rowCount();
```

borrar — DELETE

```php
$st = $pdo->prepare(
  'DELETE FROM productos WHERE id = ?');
$st->execute([$id]);
```

Marcadores con nombre (:n) o posicionales (?): elige uno y sé coherente

07

PHP + Docker

Web y base de datos en contenedores, conectadas con PDO



Definiciones: PHP + Docker

Imagen

Plantilla de solo lectura con el sistema y los programas necesarios, por ejemplo php:8.5-apache.

Contenedor

Instancia en ejecución de una imagen: un proceso aislado con su propio sistema de archivos.

Dockerfile

Archivo de texto con las instrucciones para construir una imagen personalizada.

Docker Compose

Herramienta que define y arranca varios contenedores juntos desde un archivo compose.yaml.

Volumen

Almacenamiento que gestiona Docker para que los datos sobrevivan aunque se borre el contenedor.

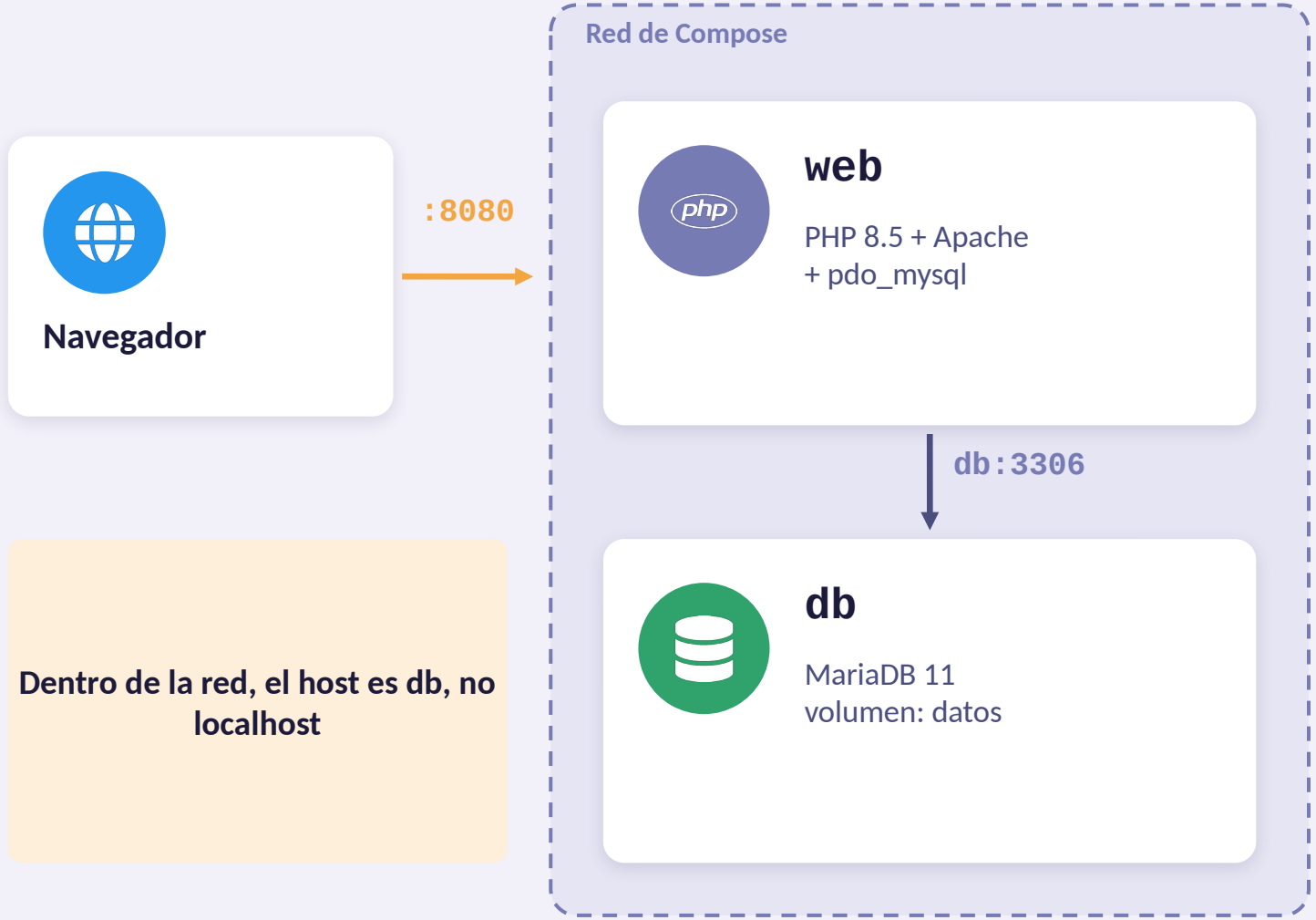
Variable de entorno

Valor de configuración que recibe el programa desde fuera del código, como las credenciales del .env.

La arquitectura del proyecto

tienda/

```
tienda/
├── compose.yaml
├── Dockerfile
├── .env              # NO a Git
├── .env.example
├── sql/
│   └── init.sql
└── src/
    ├── index.php
    ├── productos.php
    └── inc/
        └── db.php
```



Dentro de la red, el host es db, no localhost

Dockerfile y compose.yaml

Dockerfile

```
FROM php:8.5-apache-bookworm
RUN docker-php-ext-install pdo_mysql
```

.env

```
MARIADB_DATABASE=tienda
MARIADB_USER=app
MARIADB_PASSWORD=secreto
MARIADB_ROOT_PASSWORD=rootsecreto
```

compose.yaml

```
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html
    env_file: .env
    depends_on:
      - db
  db:
    image: mariadb:11
    env_file: .env
    volumes:
      - datos:/var/lib/mysql
      - ./sql:/docker-entrypoint-initdb.d
volumes:
  datos:
```

Paso a paso: qué hace docker compose up

QUÉ ES

docker compose up lee compose.yaml y crea todo lo que describe: imágenes, red, volúmenes y contenedores, en el orden necesario.

CÓMO FUNCIONA

1. Construye la imagen de web a partir del Dockerfile
2. Descarga mariadb:11 si todavía no la tienes
3. Crea la red del proyecto y el volumen datos
4. Arranca db y luego web, con las variables del .env

ERROR TÍPICO

Cambiar el .env y no ver el cambio: el usuario de la base de datos solo se crea la primera vez. Para empezar de cero: docker compose down -v.

terminal

```
# Desde la carpeta del proyecto
docker compose up -d --build

# Comprueba que los dos servicios están en marcha
docker compose ps

# Si algo falla, mira qué dice cada servicio
docker compose logs db
```

SALIDA RESUMIDA DE PS

```
NAME            SERVICE   STATUS
tienda-db-1     db        Up
tienda-web-1    web       Up
```

Conectar PHP con el contenedor de MariaDB

src/inc/db.php

```php
<?php
declare(strict_types=1);

$dsn = 'mysql:host=db;dbname=' . getenv('MARIADB_DATABASE')
     . ';charset=utf8mb4';

$pdo = new PDO($dsn,
    getenv('MARIADB_USER'),
    getenv('MARIADB_PASSWORD'),
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]
);
```

sql/init.sql

```sql
CREATE TABLE productos (
  id INT AUTO_INCREMENT
     PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  precio DECIMAL(8,2) NOT NULL
);

INSERT INTO productos
  (nombre, precio) VALUES
  ('Teclado', 29.90),
  ('Ratón', 14.50);
```

terminal

```
docker compose up -d --build
# abre http://localhost:8080/productos.php
```

init.sql solo se ejecuta con el volumen vacío. Para repetirlo: docker compose down -v

Errores típicos y cómo resolverlos

Mensaje	Causa probable	Solución
`could not find driver`	Falta pdo_mysql en la imagen	Añádelo al Dockerfile y usa up -d --build
`Connection refused`	Host localhost, o db aún arrancando	Usa host=db y espera unos segundos
`Access denied for user`	Credenciales distintas a las del .env	Revisa .env; si cambió, down -v
`Unknown database`	El volumen ya existía al crear la base	docker compose down -v y vuelve a subir
`headers already sent`	Salida antes de session_start o header	Muévelos al principio del archivo

Primero mira: docker compose ps · docker compose logs db · docker compose logs web